

### Q1. True / False Questions

1. There are **not** many cases where you need to store data locally on users' devices instead of using remote cloud servers or APIs. (      )
2. To save files to disk, combine the path\_provider plugin with the dart:io library. (      )
3. The dart:io library provides a platform-agnostic way to access commonly used locations on the device's file system. (      )
4. A directory for the app to store files that only it can access. The system clears the directory only when the app is deleted. On iOS, this corresponds to the `NSDocumentDirectory`. On Android, this is the `AppData` directory. (      )
5. Once you know where to store the file, create a reference to the file's full location. You can use the `File` class from the path\_provider library (      )
6. If you want to load content from a txt, csv or json file that found in the assets folder (or another folder in the root project) we using **rootBundle** class from **services.dart** library. (      )
7. **sqflite** in flutter are used to store data locally in key-value pairs in android and ios apps. (      )
8. Don't use `sharedpreferences` to save complex data. It is useful to save simple data. (      )
9. To save data in `sharedpreferences` in flutter declare an object of `SharedPreferences` class. Initialize the object and call the getter methods to save data. (      )
10. To save **string** value in `sharedpreferences` call the **setString()** method and provide the key and value. (      )
11. To retrieve or read data from `sharedpreferences` in flutter we have to use setter methods of `sharedpreferences` class. (      )
12. To delete or remove data from `sharedpreferences` in flutter we have to use **remove()** method. We have to provide only the key to delete the record (both the key and the value corresponding to that key). This method can delete all four types of data by simply taking the key. (      )
13. **containsKey()** method takes key as value and checks if the key is present in `sharedpreferences`. If the key is present it will return false else it will return true. (      )
14. To store some simple data persistently, you can use the **shared\_preferences** plugin. Data is persisted to disk asynchronously so you need to use **Future, async and await**. (      )



15. If you are writing an app that needs to persist and query large amounts of data on the local device, consider using a key-value store instead of a local file or database. (    )
16. In general, databases provide faster inserts, updates, and queries compared to other local persistence solutions. (    )
17. Define the path to the database file using `getDatabasesPath()` from the `path` package, combined with the `join` function from the `sqlite` package. (    )
18. To insert a data into a table in `sqlite` database. This involves two steps: Convert the data into a `Map` and Use the `add` method to store the `Map` in the table. (    )
19. Always use `whereArgs` to pass arguments to a `where` statement. This helps safeguard against SQL injection attacks. (    )

### Additional Lecture Based Statements

Mark each statement True or False. These additional items are taken from the concepts, APIs, code patterns, and engineering rules taught in Lecture 9.



21. A value stored only in a local variable inside build is not expected to survive the next rebuild. ( )
22. A State field can survive ordinary rebuilds while its State object remains mounted. ( )
23. Process termination can remove in-memory values even when the last screen looked correct. ( )
24. Persistent storage is selected when a value must be reconstructed after a later app launch. ( )
25. Uninstalling an application commonly removes its private local storage unless an external backup restores it. ( )
26. A cache can be useful without being authoritative; its freshness and invalidation policy must be understood. ( )
27. DateTime values usually need an explicit serialization format before being stored as text or JSON. ( )
28. A Boolean can be serialized as a Boolean value or as a documented representation such as 0/1, but the read path must agree. ( )
29. An absent preference key and a stored zero value are different states when zero is a valid value. ( )
30. A successful SQLite query that returns zero rows is a valid empty state, not automatically a database failure. ( )
31. A missing local file can represent first launch when the feature defines that meaning explicitly. ( )
32. Loading persistent data in initState can establish the initial source for a stateful screen. ( )
33. A State object can be unmounted while an asynchronous storage read is suspended. ( )
34. A mounted check is appropriate before applying an awaited persistence result to the UI. ( )
35. SharedPreferencesAsync reads from the platform-backed preference store rather than relying on a permanently cached value. ( )
36. A cached preferences API can return stale data if another actor changes the underlying store and the cache is not reloaded. ( )
37. SQLite is useful when data needs structured rows, filtering, ordering, and relationships. ( )
38. A file is often a natural choice for raw text, serialized JSON, or other document-like content. ( )
39. Preferences are appropriate for small key-value settings rather than arbitrary relational queries. ( )
40. A Flutter client should not store server database credentials in preferences merely because preferences persist. ( )
41. On mobile, sqflite normally stores a database in an application-controlled local database location. ( )



42. A web target can have different file and SQLite capabilities from Android or iOS, so the storage plan must name its platform assumptions. ( )
43. A repository boundary can keep UI code independent from whether data comes from preferences, SQLite, files, or a remote source. ( )
44. Widgets should not need to know SQL syntax in order to render a list of domain objects. ( )
45. A toMap method should produce keys and value types that agree with the database or JSON contract. ( )
46. A fromMap or fromJson method should validate required fields and normalize values at the serialization boundary. ( )
47. A SQL where clause should bind user values through whereArgs rather than concatenate them into SQL text. ( )
48. Using whereArgs separates a values such as an id from the SQL structure and reduces injection risk. ( )
49. An update that omits its where clause can modify every row, so the target boundary must be explicit. ( )
50. A delete operation should identify the intended row with a stable key and a matching bound argument. ( )
51. ConflictAlgorithm.replace is a policy decision and can overwrite a conflicting row; it is not a universal safest default. ( )
52. A transaction is appropriate when several database changes must succeed or fail as one logical unit. ( )
53. A batch groups operations for efficiency but should not automatically be described as a transaction without checking the API semantics. ( )
54. A fresh query after a write gives stronger evidence of stored data than changing a local list optimistically and stopping there. ( )
55. A controller used by a form is a resource owned by the State object and should be disposed there. ( )
56. A JSON decoder can parse syntactically valid JSON that still violates the application schema. ( )
57. Schema validation should distinguish a missing required field from an optional field whose value is null. ( )
58. A local-only feature and an offline-first feature have different synchronization and conflict responsibilities. ( )
59. Offline-first design needs a policy for stale data, pending writes, conflicts, and later synchronization. ( )
60. A directory path should be joined with a filename rather than assembled with platform-specific separators by hand. ( )
61. A path provider gives a platform-appropriate directory, but the application still decides which file belongs there. ( )
62. Temporary files are not automatically suitable as long-term application records. ( )



- 63. A missing counter file can be mapped to a defined initial value before the next write. ( )
- 64. Parsing persisted text should handle malformed input as a controlled error or recovery path. ( )
- 65. A restart test is stronger evidence of persistence than seeing the value immediately after setState. ( )
- 66. SharedPreferencesAsync can store supported primitive types, but the read type should match the write contract. ( )
- 67. Calling remove for one preference key should not be described as clearing every unrelated preference unless the API call actually does that. ( )
- 68. A clear operation should be used with awareness of its scope because it can remove more keys than a targeted remove. ( )
- 69. A cached preference instance may need reload or recreation before it can observe an external update. ( )
- 70. A database schema version gives an upgrade path when the structure changes between app releases. ( )
- 71. onUpgrade is the place to apply the documented migration from an older schema to a newer one. ( )
- 72. A migration should preserve or transform existing user data deliberately rather than silently dropping columns. ( )
- 73. UI loading, empty, error, and data states should reflect storage evidence rather than one generic blank screen. ( )
- 74. A persistence layer should report failure instead of showing a success message before an awaited write completes. ( )
- 75. Deleting a row from a local database does not imply that a remote copy has also been deleted. ( )
- 76. An offline-first repository may need both a local authoritative view and a remote synchronization status. ( )
- 77. A storage decision should consider lifetime, query shape, data size, security, platform support, and recovery needs. ( )
- 78. A persistence test should verify the restored value after a new startup path, not only immediately after the write callback. ( )
- 79. A repository can expose domain-level operations while hiding whether the current implementation uses a preference, file, SQLite, or remote source. ( )
- 80. If the UI cannot distinguish loading, empty, error, and data, it may report a misleading persistence result even when the storage layer is correct. ( )



### Code and Screen True / False

If we have the following code fragment, mark the statement that follows True or False.

81. If we have the following code fragment:

```
FloatingActionButton(  
  onPressed: () => _showForm(null),  
  child: const Icon(Icons.add),  
)
```

A local SQLite query can return zero rows successfully, so the UI should distinguish empty from error. ( )

82. If we have the following code fragment:

```
setState(() {  
  _journals.add({'title': 'Study Plan', 'description': 'Review SQLite CRUD'});  
});
```

A FAB callback can open a create form while keeping the list route available for refresh after return. ( )

83. If we have the following code fragment:

```
@override  
void initState() {  
  super.initState();  
  _retrieveName();  
}  
Future<void> _retrieveName() async {  
  final value = await _prefs.getString(_nameKey) ?? '';  
  if (!mounted) return;  
  setState(() => savedName = value);  
}
```

After create completes, a fresh query is evidence that the new row is actually stored. ( )

84. If we have the following code fragment:

```
await prefs.setBool('dark_mode', true);  
final dark = await prefs.getBool('dark-mode') ?? false;
```

After update completes, querying the target id again verifies the edited values rather than trusting the form. ( )



85. If we have the following code fragment:

```
final file = await _dataFile();
final data = await file.exists()
    ? await file.readAsString(encoding: utf8)
    : '';
if (!mounted) return;
setState(() => _content = data);
```

After delete and refresh, a success message is truthful only after the awaited operation has completed. ( )

86. If we have the following code fragment:

```
final data = await file.exists()
    ? await file.readAsString()
    : '';
```

A saved preference must be read back into State before the restored name can be shown after relaunch. ( )

87. If we have the following code fragment:

```
final decoded = jsonDecode(raw);
if (decoded is! Map<String, dynamic>) {
    throw const FormatException('Expected a JSON object');
}
```

A process restart can restore persistent data only when startup code reads the correct store. ( )

88. If we have the following code fragment:

```
setState(() => loading = true);
final data = await repository.loadAll();
if (!mounted) return;
setState(() { loading = false; items = data; });
```

A file read should map an absent file to the feature's defined first-run state, not to malformed JSON. ( )

## Q2. Multiple Choice Questions

1. The persistence options you have on Flutter include:



- A. File storage (Read and write files).
  - B. Key-Value Store (SharedPreferences).
  - C. Local database using SQLite
  - D. All of the mentioned
2. To save files to disk, combine the `path_provider` plugin with the ..... library
  - A. `dart:io`
  - B. `dart:async`
  - C. A and B
  - D. None of the mentioned
3. The ..... package provides a platform-agnostic way to access commonly used locations on the device's file system.
  - A. `path_provider`
  - B. `dart:io`
  - C. `dart:async`
  - D. All of the mentioned
4. The `path_provider` plugin currently supports access to ..... :
  - A. One file system location (Temporary directory)
  - B. One file system location (Documents directory)
  - C. Two file system locations (Temporary and Documents directory)
  - D. None of the mentioned
5. A ..... that the system can clear at any time.
  - A. Documents directory (cache)
  - B. Temporary directory (cache)
  - C. A and B.
  - D. None of the mentioned
6. A directory for the app to store files that only it can access. The system clears the directory only when the app is deleted. On iOS, this corresponds to the ..... On Android, this is the ..... directory.



- A. AppData, NSDocumentDirectory
  - B. NSCachesDirectory, getCacheDir
  - C. NSDocumentDirectory, AppData
  - D. None of the mentioned
7. A temporary directory (cache) that the system can clear at any time. On iOS, this corresponds to the ..... On Android, this is the value that ..... returns.
- A. AppData, NSDocumentDirectory
  - B. NSCachesDirectory, getCacheDir
  - C. NSDocumentDirectory, AppData
  - D. None of the mentioned
8. Once you know where to store the file, create a reference to the file's full location. You can use the ..... class from the dart:io library.
- A. File
  - B. Path
  - C. A and B
  - D. None of the mentioned
9. To write some data to the file. The counter is an integer, but is written to the file as a string using the '\$counter' syntax
- A. file.write('\$counter')
  - B. file.writeAsInt('\$counter')
  - C. file.writeAsString('\$counter')
  - D. None of the mentioned
10. If you have some data on disk (The counter is an integer) , you can read it. Once again, use the File class and use ..... function.
- A. file.readAsString()
  - B. file.readAsInt()
  - C. info file.read()
  - D. None of the mentioned



11. ....in flutter are used to store data locally in key-value pairs in android and ios apps. We might have to save data locally while developing applications. Data like login credentials or a flag etc.
- A. Sqflite
  - B. SharedPreferences
  - C. Files
  - D. None of the mentioned
12. Using sharedPreferences we can save multi types of values such as .
- A. int, string,
  - B. double, bool,
  - C. stringList.
  - D. All of the mentioned
13. To implement and use sharedPreferences in flutter follow the steps below
- A. Add shared\_preferences plugin or dependency to the project.
  - B. Import the shared preferences package :- import 'package:shared\_preferences/shared\_preferences.dart';
  - C. Implement the code as per your requirement like saving, retrieving, deleting, etc.
  - D. All of the mentioned
14. To save **string** value in sharedPreferences call the ..... method and provide the key and value.
- A. setString()
  - B. setStringList()
  - C. set()
  - D. None of the mentioned
15. To save the **boolean** value in sharedPreferences call the ..... method and provide the key and value.
- A. setString()
  - B. setStringList()
  - C. set()
  - D. setBool()
16. To save a string list value in sharedPreferences call the **setStringList()** method and provide the key and value. The next code snippet will add list( ["pizza", "burger", "sandwich"] ) as a value to the key **foodList**



```
A. prefs = await SharedPreferences.getInstance();  
   prefs.setStringList("foodList", ["pizza", "burger", "sandwich"]);  
  
B.  prefs = await SharedPreferences.getInstance();  
   prefs.setStringList("foodList", {"pizza", "burger", "sandwich"});  
  
C.  prefs =  SharedPreferences.getInstance();  
   prefs.setStringList("foodList", ["pizza", "burger", "sandwich"]);
```

D. None of the mentioned

17. To retrieve or read **int** value from sharedpreferences call ..... method and provide the key.

- A. get()
- B. setInt()
- C. set()
- D. getInt()

18. To retrieve or read **double** value from sharedpreferences call ..... method and provide the key.

- A. get()
- B. getFloat ()
- C. getDouble()
- D. getInt()

19. To retrieve or read **string list** value from sharedpreferences call ..... method and provide the key.

- A. getString()
- B. getStringList()
- C. get()
- D. getList()

20. To check if a key or record is present in shared preferences we have to use ..... method. This methods takes key as value and checks if the key is present in sharedpreferences.

- A. contains()
- B. containsVal()



- C. `containsKey()`
  - D. `findesKey()`
- 21. To reset and remove any saved data in sharedpreferences call the ..... method without provide the key and value
  - A. `remove()`
  - B. `clear()`
  - C. A and B
  - D. None of the mentioned
- 22. Flutter apps can make use of the SQLite databases via the ..... plugin available on pub.dev.
  - A. `sqflight ()`
  - B. `sqlite ()`
  - C. `sqlFlutter()`
  - D. None of the mentioned
- 23. The ..... package provides classes and functions to interact with a SQLite database.
  - A. `sqlite`
  - B. `path`
  - C. A and B
  - D. None of the mentioned
- 24. The ..... package provides functions to define the location for storing the database on disk.
  - A. `sqlite`
  - B. `path`
  - C. A and B
  - D. None of the mentioned
- 25. Define the path to the database file using ..... from the `sqlite` package, combined with the `join` function from the `path` package.
  - A. `getDatabasesPath()`
  - B. `getPath()`
  - C. `getDatabases()`



- D. None of the mentioned
26. Before reading and writing data to the database, open a connection to the database. Open the database with the ..... function from sqlite.
- A. openDatabasePath()
  - B. openDatabase()
  - C. getOpenDatabases()
  - D. None of the mentioned
27. To insert a data into a table in sqlite database. This involves:
- A. Use the insert() method to store the Map in the table.
  - B. Convert the data into a List and Use the insert() method to store the Map in the table.
  - C. Convert the data into a Map and Use the insert() method to store the Map in the table.
  - D. None of the mentioned
28. In addition to inserting and updating information about Dogs, you can also remove dogs from the database. To delete data, use the ..... method from the ..... library.
- A. delete(), path
  - B. delete(), sqlite
  - C. del(), sqlite
  - D. None of the mentioned
29. .... is a fast relational database that can be used to store data offline for mobile applications.
- A. Hive
  - B. Sembast
  - C. SQLite
  - D. None of the mentioned
30. In flutter is used to store data locally in android and ios apps
- A. Txt files
  - B. SharedPreferences
  - C. SQLite
  - D. All of the mentioned



### Additional Lecture Based Multiple Choice Questions

Choose the one best answer. The questions cover only storage selection, serialization, SQLite, preferences, files, lifecycle, migration, security, architecture, and testing topics taught in Lecture 9.

31. A counter appears correct until the app process is terminated and relaunched. What storage is missing?
  - A. Persistent storage plus startup restoration
  - B. A second setState call only
  - C. A different AppBar color
  - D. A new BuildContext
  
32. A value is created as a local variable inside build. What is the main limitation?
  - A. It can be recreated and lost on rebuilds
  - B. It automatically survives uninstall
  - C. It is stored in SQLite
  - D. It is shared with the server
  
33. Which value is a natural candidate for a small key-value preference?
  - A. A user-selected theme mode
  - B. A table of thousands of journal rows with joins
  - C. A large binary video
  - D. A relational query plan
  
34. A journal app needs filtering, ordering, and updating individual rows by id. Which store fits best?
  - A. SQLite
  - B. A single preference key
  - C. A TextEditingController
  - D. The widget tree only
  
35. A notes feature stores one document as JSON text and reads it back as a whole. Which store is a natural fit?
  - A. A file
  - B. A route name
  - C. A TabController
  - D. A single Boolean preference only



36. A user preference must be read after a process restart. Which design is required?
- A. Write it to a persistent preference store and read it during startup
  - B. Keep it only in a local build variable
  - C. SetState after every frame
  - D. Store it in BuildContext
37. A stored username is absent on first launch. What is a robust interpretation?
- A. Show an explicit first-run or empty state
  - B. Treat it as a database corruption automatically
  - C. Display the password hash
  - D. Create a random username silently
38. A SQLite query succeeds but returns an empty list. What should the UI show?
- A. A truthful empty state
  - B. A loading spinner forever
  - C. A fabricated journal
  - D. A SQL exception automatically
39. A file does not exist the first time the app runs. What should a JSON repository do if first-run is valid?
- A. Return a defined empty/first-run result
  - B. Read a random path
  - C. Assume the file contains valid JSON
  - D. Write database credentials into it
40. Why check mounted after awaiting a database read in a StatefulWidget?
- A. The screen may have been removed before the read completed
  - B. It converts rows to maps
  - C. It creates a schema version
  - D. It makes a query transactional
41. A preference value changed outside a cached reader, but the UI still shows the old value. What is likely?
- A. The reader cache was not reloaded or bypassed



- B. SQLite has no rows
  - C. The file path is too long
  - D. The widget has a Hero tag
42. Which boundary best isolates a Flutter screen from storage implementation details?
- A. A repository/service interface returning domain data
  - B. SQL strings embedded in every build method
  - C. A global BuildContext
  - D. A TextEditingController used as a database
43. A Dog model needs to be inserted into SQLite. What should toMap provide?
- A. Column names mapped to database-compatible values
  - B. A Widget tree
  - C. A route result
  - D. An HTTP status only
44. Which query is safer for a user-provided id?
- A. `db.query("dogs", where: "id = ?", whereArgs: [id])`
  - B. `db.rawQuery("... id = $id")`
  - C. `db.query("dogs", where: "id = " + id)`
  - D. `BuildContext.read(id)`
45. Why is string concatenation unsafe in SQL conditions?
- A. Untrusted input can alter the SQL structure
  - B. It always makes queries slower than a file
  - C. It prevents all updates
  - D. It changes a Future into a Stream
46. An update must affect exactly one journal row. Which detail is essential?
- A. A where clause on the stable id with matching whereArgs
  - B. Omitting where to simplify the code
  - C. Using the row title as SQL text
  - D. Updating every row then filtering in the widget



47. When are transactions most appropriate?
- A. When several database changes must be atomic as one logical operation
  - B. When reading one preference key
  - C. When rendering a Text widget
  - D. When selecting an image
48. What is a cautious interpretation of `ConflictAlgorithm.replace`?
- A. A declared conflict policy that may replace an existing row
  - B. A guarantee that no data can be overwritten
  - C. A JSON validation method
  - D. A lifecycle check
49. A list is updated locally after insert without reading the database. What evidence is missing?
- A. A fresh query or equivalent confirmed persistence evidence
  - B. A new AppBar
  - C. A second controller
  - D. A different route name
50. Which code owns form controllers in a `StatefulWidget`?
- A. The State object, including disposal
  - B. The SQLite table
  - C. The route name
  - D. The JSON decoder
51. A string is valid JSON but lacks the required title field. What should the model boundary do?
- A. Reject it or return a controlled validation error
  - B. Invent a title silently in every UI widget
  - C. Treat it as a successful complete note
  - D. Delete the database
52. Which serialization rule is safest for an optional date?
- A. Use a documented nullable representation and parse it consistently



- B. Assume every date is a Widget
  - C. Write a `DateTime` object directly to a text file with no format
  - D. Ignore null values without a policy
53. What does a schema version enable?
- A. A deliberate migration path between database structures
  - B. Automatic server synchronization
  - C. Password hashing
  - D. A replacement for `whereArgs`
54. Where should an older database be upgraded when the app opens a newer schema?
- A. The database on `Upgrade migration` callback
  - B. A `ListView.builder` item
  - C. The `AppBar`
  - D. The image picker
55. A new column is added to an existing database. What is a responsible migration concern?
- A. Preserve existing rows and define a safe value for the new column
  - B. Drop every old row automatically
  - C. Ignore the schema version
  - D. Move the column into `BuildContext`
56. Which lifecycle best describes a value held in State but not persisted?
- A. It can survive rebuilds but not necessarily process termination
  - B. It survives uninstall by definition
  - C. It is always shared with `SQLite`
  - D. It is immutable across all app launches
57. A file path should be built for Android and iOS. Which approach is strongest?
- A. Use a platform directory from `path_provider` and join the filename with `path.join`
  - B. Hard-code one slash format everywhere
  - C. Use a SQL `where` clause
  - D. Store the path in a widget key only



58. Why is a temporary image/file path a poor long-term database value?
- A. The temporary location may disappear or be inaccessible later
  - B. Temporary files are always remote URLs
  - C. The path automatically contains a password
  - D. JSON cannot contain paths
59. A persistence read throws because the stored text is malformed. What should the UI normally receive?
- A. A controlled error/recovery state
  - B. A guaranteed valid model
  - C. A random default without logging
  - D. A successful delete message
60. Which test best demonstrates persistence?
- A. Write, terminate/restart the process, read during startup, and verify the restored value
  - B. Set a field and rebuild once
  - C. Tap Save and inspect the same TextField
  - D. Open a dialog without writing
61. Which API scope is appropriate for removing one preference?
- A. remove with the specific key
  - B. clear when only one key should change
  - C. SQLite delete without a table
  - D. Navigator.pop
62. Why should clear be used carefully?
- A. It may remove all keys in the relevant preference store
  - B. It changes only one character
  - C. It creates a database migration
  - D. It updates a remote server automatically
63. A cache-backed preference reader must observe external changes. What operation may be required?
- A. Reload the cache or read through a non-cached API



- B. Call `setState` only
  - C. Use a `Hero` tag
  - D. Drop the `SQLite` database
64. Which data belongs in server-side configuration rather than local preferences?
- A. Credentials used by an API to access a database
  - B. A non-sensitive theme selection
  - C. A local sort choice
  - D. A last-opened tab index
65. Which architecture supports later remote synchronization for an offline-first feature?
- A. A repository that separates local data, remote operations, sync status, and conflict policy
  - B. A single `TextField` with no storage
  - C. Direct `SQL` from every widget
  - D. A route result only
66. What is the main difference between local-only and offline-first?
- A. Offline-first anticipates later synchronization and conflicts with a remote source
  - B. Local-only always needs a server
  - C. Offline-first cannot use `SQLite`
  - D. They have identical responsibilities
67. A screen reads a stored name in `initState` and calls `setState` after `await`. What must it check?
- A. mounted before the post-await update
  - B. The `Hero` tag before every `setState`
  - C. The route name as an integer
  - D. The `SQL` password in the widget
68. Which result should a repository return for a missing JSON file when the product defines first-run as empty?
- A. A distinct empty/first-run outcome
  - B. A fake record
  - C. A successful remote synchronization
  - D. A password hash



69. Why keep SQL out of widgets?
- A. It keeps presentation separate from persistence and makes storage testable
  - B. SQL can only run in initState
  - C. Widgets cannot display lists
  - D. It makes all writes synchronous
70. A query returns rows as maps. Where should map-to-model conversion normally happen?
- A. At the repository/model boundary
  - B. Inside every ListTile independently
  - C. Inside AppBar
  - D. Inside NavigatorObserver
71. What does a fresh GET after a remote write prove more strongly than a local list append?
- A. The server-backed state contains the changed record
  - B. The form was opened
  - C. The button received focus
  - D. The local controller is not disposed
72. A delete confirmation is cancelled. What should happen to persistent data?
- A. No delete request should be issued
  - B. Delete every row
  - C. Clear all preferences
  - D. Drop the database schema
73. A file contains JSON for a LocalNote. What should be checked after decoding?
- A. Required fields and value types before constructing the model
  - B. Only that the file extension is .json
  - C. Only that a Text widget can display it
  - D. Only the current route
74. Which choice best handles a stored integer that may be absent?
- A. Use a documented default only when absence is semantically allowed



- B. Treat absence as zero in every domain
  - C. Crash every first launch
  - D. Store a BuildContext
75. Why distinguish an empty SQLite query from an error?
- A. The user needs to know that storage succeeded but no matching rows exist
  - B. Empty always means a malformed schema
  - C. Errors cannot be shown in Flutter
  - D. SQLite always returns one fake row
76. Which database operation matches “update the row whose id is 4”?
- A. update with where: "id = ?" and whereArgs: [4]
  - B. update with no where clause
  - C. insert a second row with id 4 blindly
  - D. query and change only the Text widget
77. Which operation is appropriate for a new journal row?
- A. insert with a validated map
  - B. delete with whereArgs
  - C. getString
  - D. Navigator.push
78. A schema migration drops a column containing user notes without backup. What is the primary problem?
- A. Existing user data is lost by an unsafe migration
  - B. The UI has too many buttons
  - C. The route result is nullable
  - D. The preference cache is fresh
79. A local successSnackBar appears before an awaited write completes. What is wrong?
- A. The UI claims persistence before it has evidence of success
  - B. SnackBars cannot be used with SQLite
  - C. Await always deletes data
  - D. The database must be on the server



80. What is a useful test seam for persistence?
- A. A repository interface or replaceable storage service
  - B. A hard-coded SQL string in every widget
  - C. A global BuildContext
  - D. A screenshot with no state assertions
81. A stored timestamp is read on a device with a different locale. What should the model use?
- A. A stable serialized representation with deliberate timezone semantics
  - B. The localized display string as the only source of truth
  - C. A random DateTime.now on every read
  - D. The route name
82. Which statement about SharedPreferencesAsync is most useful?
- A. It avoids relying solely on a local cache for each read
  - B. It stores relational joins
  - C. It uploads values to a server automatically
  - D. It replaces all file access
83. A list is refreshed after a delete. What should the UI do if the fresh query returns zero rows?
- A. Render the empty state
  - B. Keep the deleted row as proof of success
  - C. Show loading forever
  - D. Insert a placeholder into SQLite
84. Why should an app name its authoritative home for each data item?
- A. It clarifies which source is trusted and how restoration/synchronization works
  - B. It makes all data global
  - C. It avoids serialization
  - D. It removes the need for migrations
85. What is the strongest overall storage decision criterion?
- A. Required lifetime and data behavior together with query, security, platform, and recovery needs



- B. Which API has the shortest name
  - C. Which store appears first in autocomplete
  - D. Whether the screen uses a Scaffold
86. A local note is saved as JSON but the schema later gains a required field. What is needed?
- A. A versioning/default/migration policy for old documents
  - B. A new route name only
  - C. A larger AppBar
  - D. A second controller without validation
87. Which statement about a database id is safest?
- A. Use a stable id for exact-row update/delete boundaries
  - B. Use the visible list index as a permanent id
  - C. Use a password as an id
  - D. Use BuildContext as an id
88. A repository catches malformed storage data. What should it provide to the UI?
- A. A typed error or recoverable result with a clear user-facing state
  - B. A silent success result
  - C. A raw SQL password
  - D. A duplicated write
89. A process restart restores a value from storage. Which layer performed the restoration?
- A. Startup/read logic reconstructed State from persistent data
  - B. A previous build local variable
  - C. The last route pop
  - D. The widget color
90. Which design avoids binding persistence behavior to a particular screen?
- A. A reusable repository/service consumed by the screen
  - B. SQL and file I/O inside every button callback
  - C. A static BuildContext field
  - D. A screenshot-only test



### Practical Code and Expected Output

For each item, read the code fragment first, then choose the exact stored result, visible state, or persistence evidence that follows.

91. If we have the following code fragment:

```
await insertDog(const Dog(id: 0, name: 'Fido', age: 35));  
print(await dogs());
```

After inserting Fido with age 35, what exact list is printed?

- A. [Dog{id: 0, name: Fido, age: 35}]
- B. []
- C. 42
- D. null

92. If we have the following code fragment:

```
fido = Dog(id: 0, name: 'Fido', age: 42);  
await updateDog(fido);  
print(await dogs());
```

After updating Fido to age 42, what exact list is printed?

- A. [Dog{id: 0, name: Fido, age: 42}]
- B. [Dog{id: 0, name: Fido, age: 35}]
- C. []
- D. FormatException

93. If we have the following code fragment:

```
await deleteDog(fido.id);  
print(await dogs());
```

After deleting Fido, what exact list is printed?

- A. []
- B. [Fido]
- C. true



D. counter=8

94. If we have the following code fragment:

```
final prefs = SharedPreferencesAsync();  
print(await prefs.getString('username'));
```

When the username key is absent, what does getString print?

- A. null
- B. Eng:Ruqaih Salman
- C. 0
- D. []

95. If we have the following code fragment:

```
await prefs.setString('username', 'Eng:Ruqaih Salman');  
print(await prefs.getString('username'));
```

After saving the username, what does the following read print?

- A. Eng:Ruqaih Salman
- B. null
- C. false
- D. File not found

96. If we have the following code fragment:

```
await prefs.remove(usernameKey);  
print(await prefs.containsKey(usernameKey));
```

After removing usernameKey, what does containsKey print?

- A. false
- B. true
- C. 1
- D. FormatException



97. If we have the following code fragment:

```
final contents = await file.readAsString();  
return int.parse(contents);
```

If the text file contains 7, what integer is returned?

- A. 7
- B. '7' as String
- C. File
- D. null

98. If we have the following code fragment:

```
final counter = int.parse(await file.readAsString());  
await file.writeAsString('${counter + 1}');  
print(int.parse(await file.readAsString()));
```

If the file contains 7, what is printed after incrementing and overwriting it?

- A. 8
- B. 7
- C. 0
- D. null

99. If we have the following code fragment:

```
if (!await file.exists()) return null;
```

If the JSON file does not exist, what first-run result is returned?

- A. null
- B. LocalNote(text: ")
- C. FormatException always
- D. A SQLite row



100. If we have the following code fragment:

```
final decoded = jsonDecode(raw);  
if (decoded is! Map<String, dynamic>) {  
  throw const FormatException('Expected a JSON object');  
}
```

If decoded JSON is not a map, which result follows the validation code?

- A. FormatException
- B. Success state
- C. SQLite migration
- D. Preference reset only



## Code to Screen Visual Multiple Choice

If we have the following code fragment or persistence condition, choose the exact labeled visual output A, B, C, or D.

101. If we have the following code fragment:

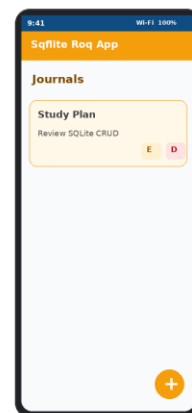
```
final data = await SQLHelper.getItems();  
if (!mounted) return;  
setState(() { _journals = data; _isLoading = false; });
```

A refreshed SQLite query returns an empty list. Which labeled screen is correct?

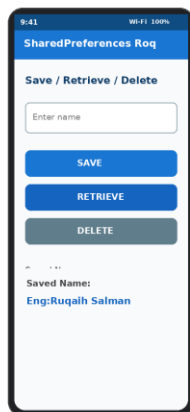
A.



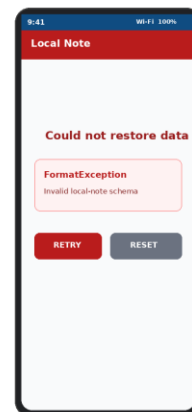
B.



C.



D.

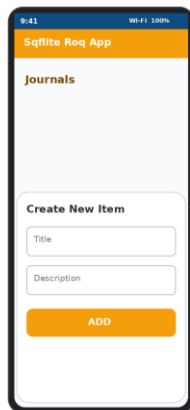


102. If we have the following code fragment:

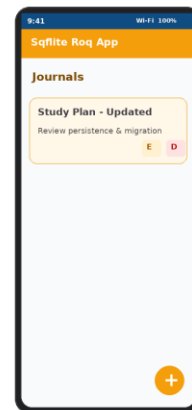
```
onPressed: () => _showForm(null)
```

The FAB callback opens a form route. Which labeled screen is on top?

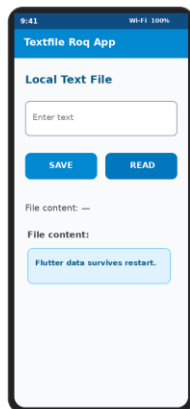
A.



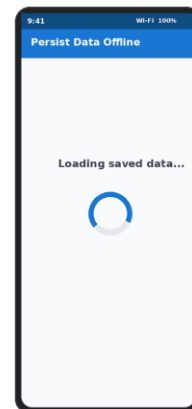
B.



C.



D.

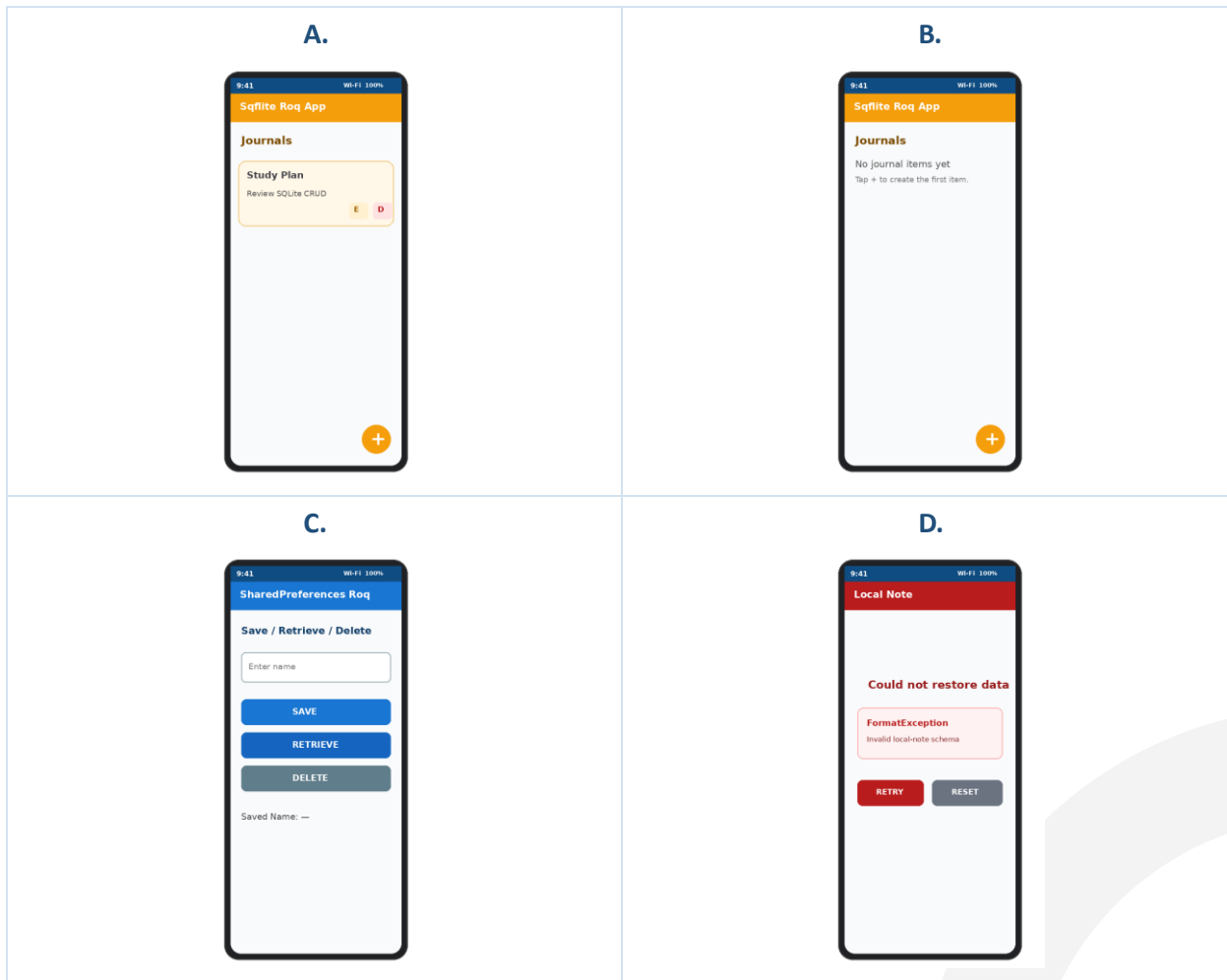


103. If we have the following code fragment:

```
await SQLHelper.createItem(title, description);  
await _refreshJournals();
```

A create operation is confirmed and the fresh query contains the new row. Which labeled screen is correct?



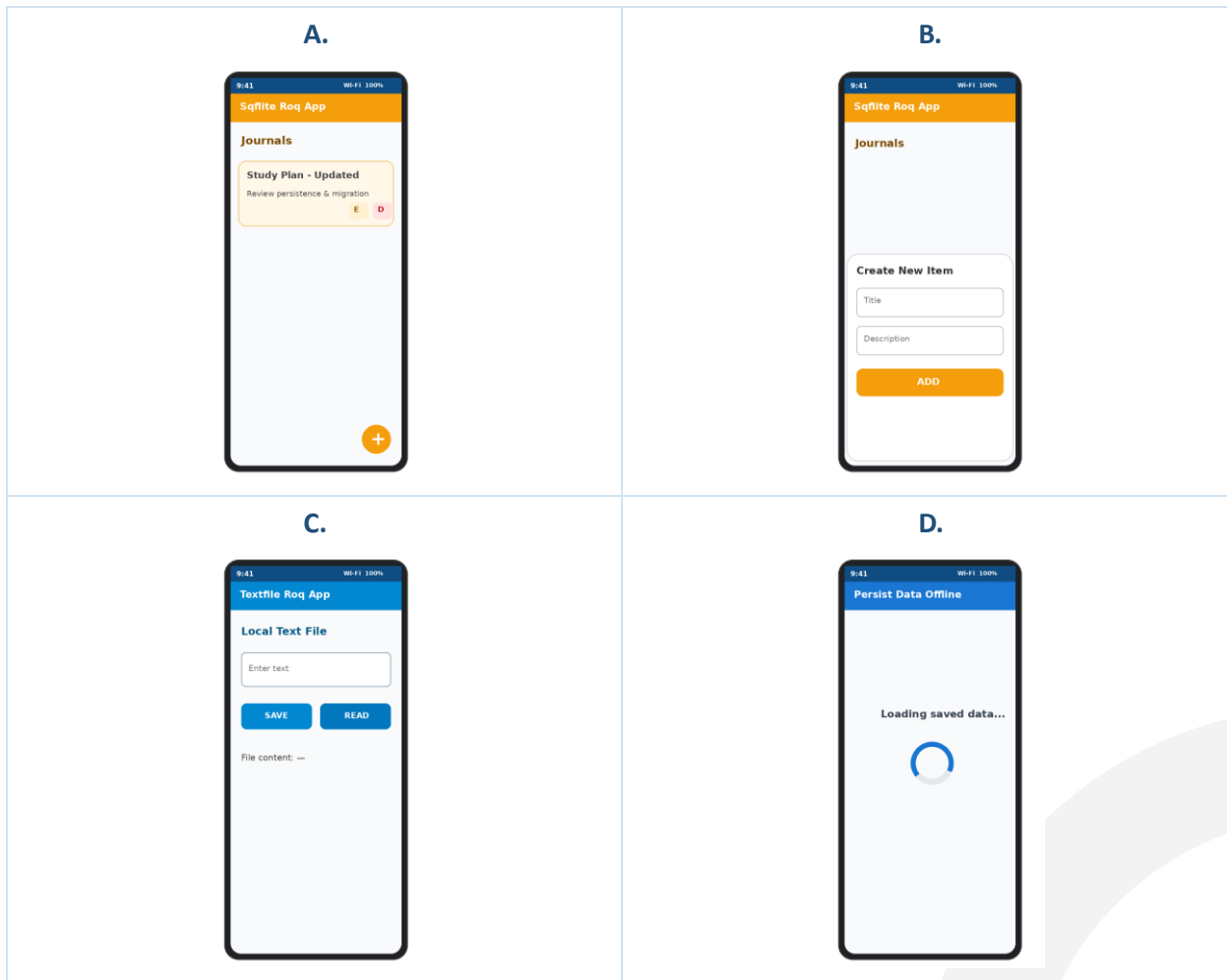


104. If we have the following code fragment:

```
await SQLHelper.updateItem(id, 'Study Plan - Updated', 'Review persistence & migration');
await _refreshJournals();
```

An update operation completes and the query returns the edited row. Which labeled screen is correct?



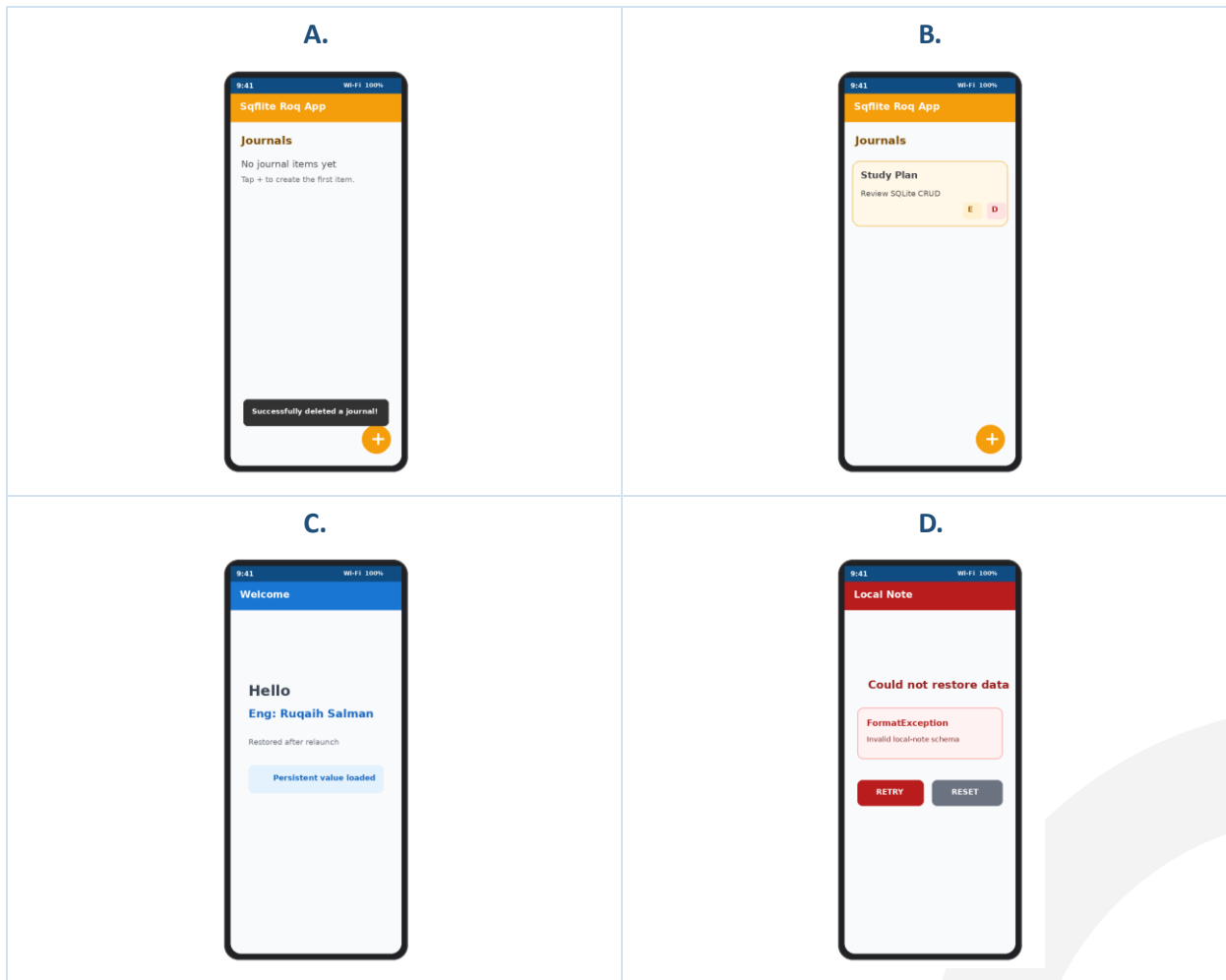


105. If we have the following code fragment:

```
await SQLHelper.deleteItem(id);  
await _refreshJournals();  
ScaffoldMessenger.of(context).showSnackBar(...);
```

A delete is confirmed, awaited, refreshed, and followed by a success message. Which labeled screen is correct?



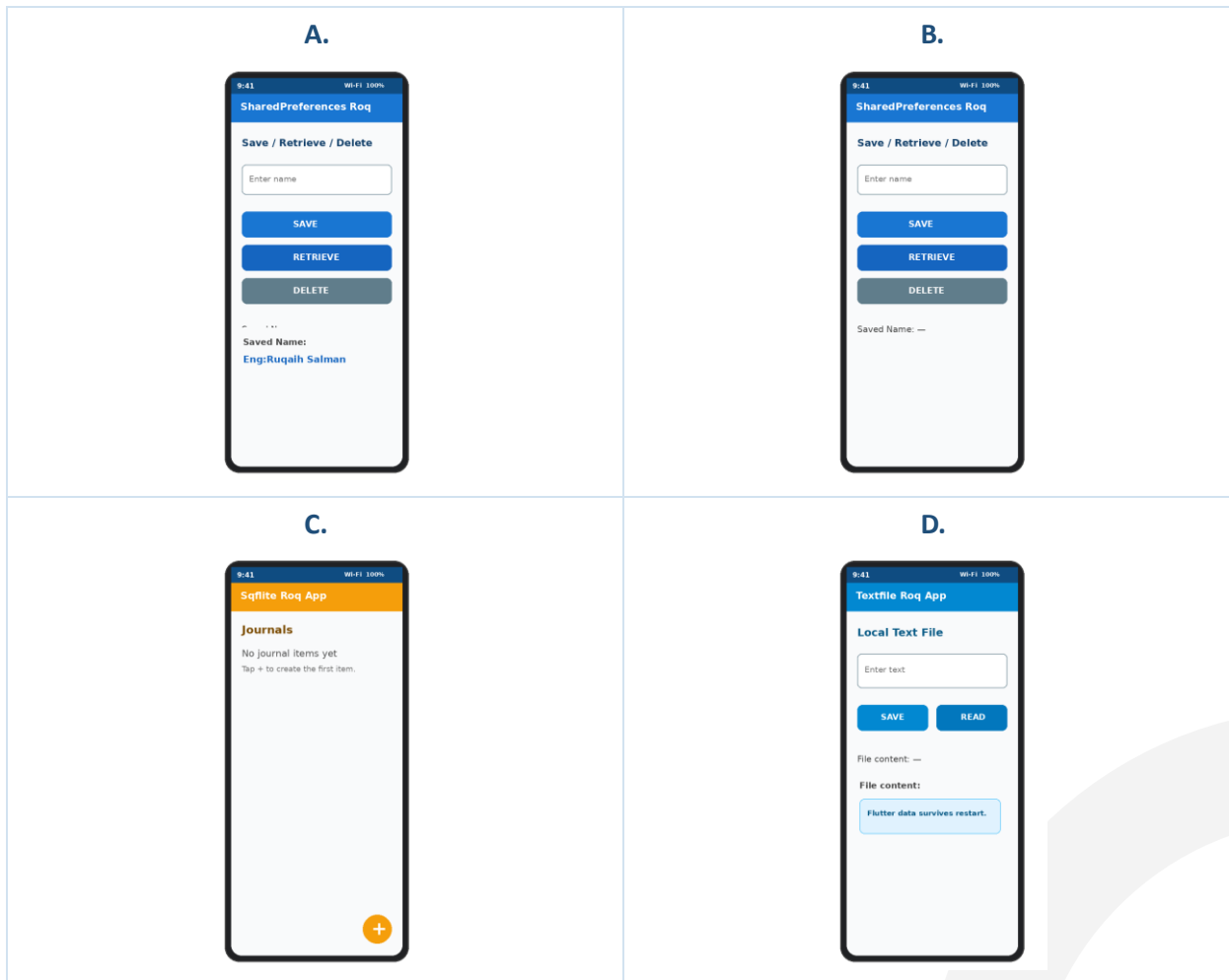


106. If we have the following code fragment:

```
final value = await _prefs.getString(_usernameKey) ?? '';  
if (mounted) setState(() => name = value);
```

A preference string is retrieved into State. Which labeled screen displays the name?



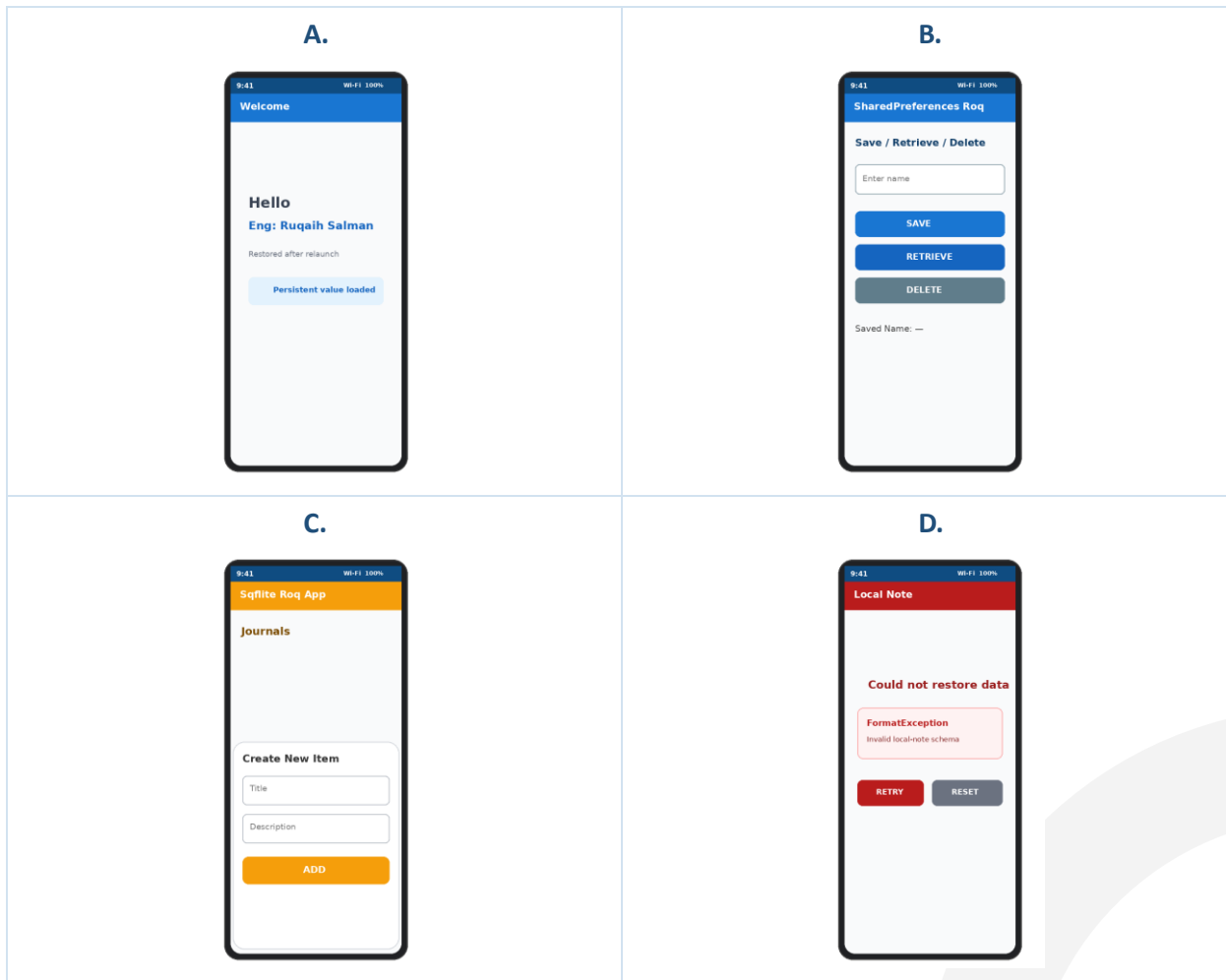


107. If we have the following code fragment:

```
@override
void initState(){ super.initState(); _retrieveName(); }
```

The app restarts and startup restoration succeeds. Which labeled screen proves the value survived?



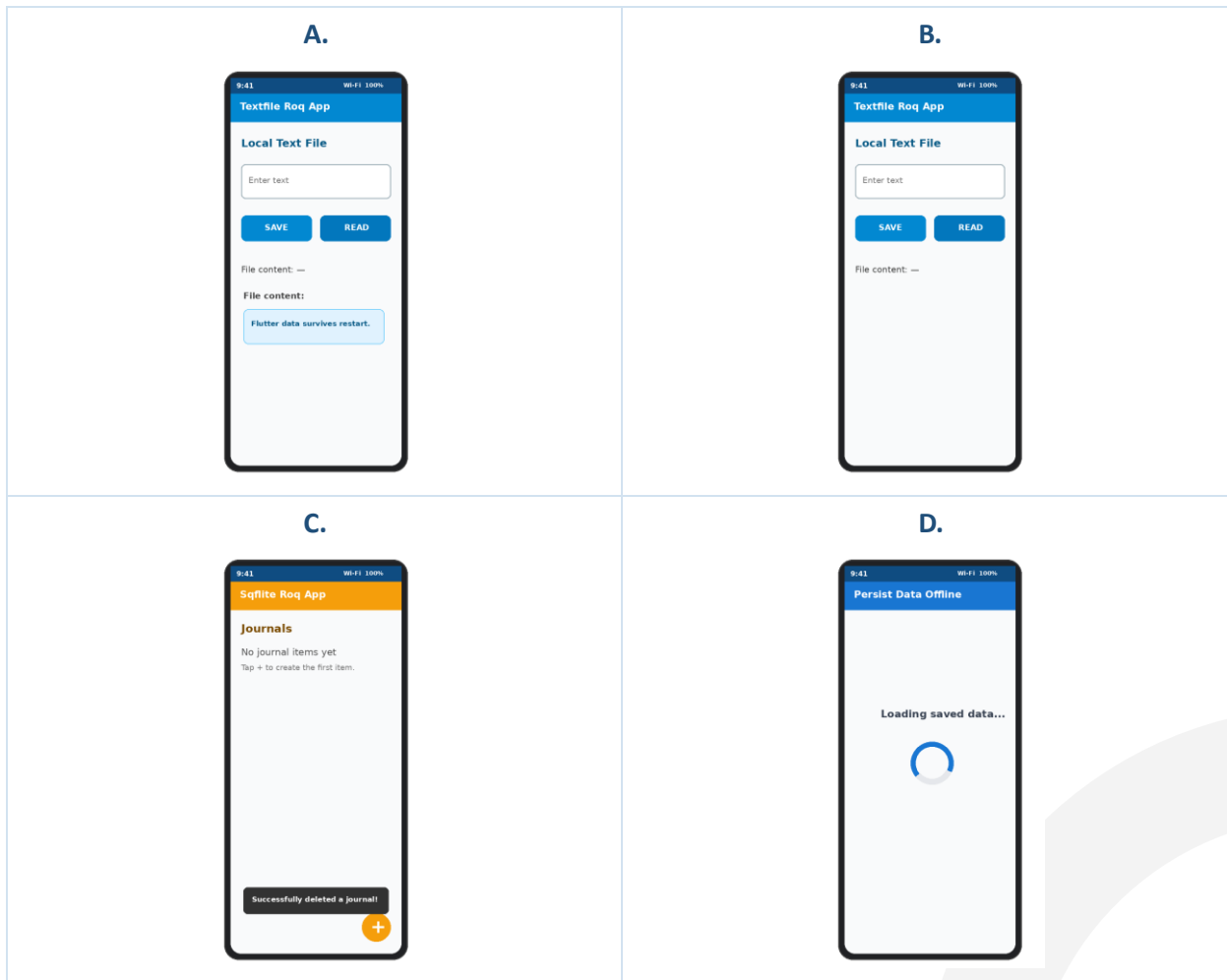


108. If we have the following code fragment:

```
final data = await file.readAsString(encoding: utf8);  
if (mounted) setState(() => _content = data);
```

An existing text file is read and assigned to `_content`. Which labeled screen is correct?



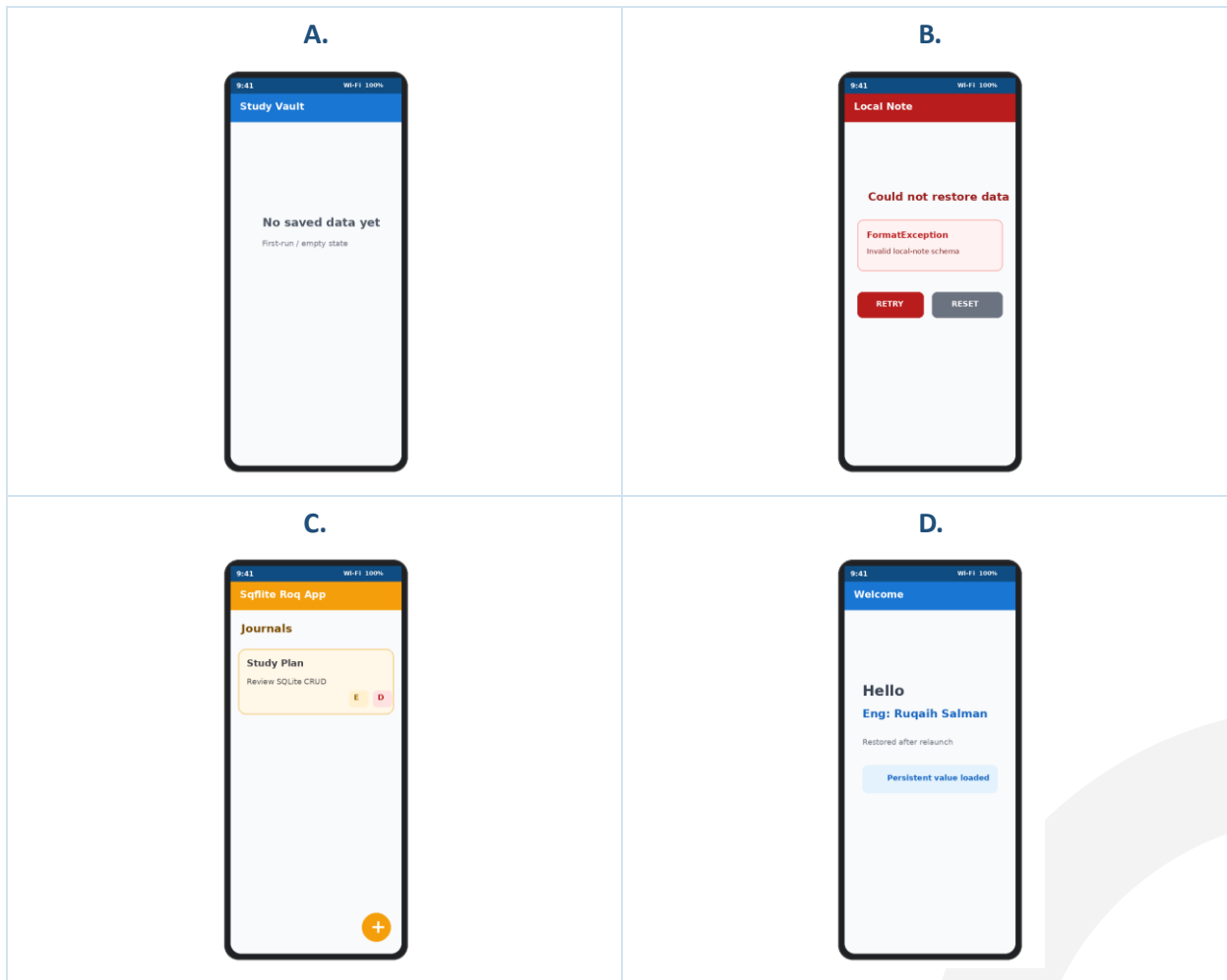


109. If we have the following code fragment:

```
if (!await file.exists()) return null;
```

A JSON file is missing and null maps to first-run state. Which labeled screen is correct?





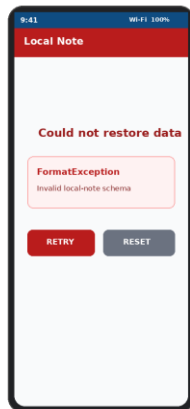
110. If we have the following code fragment:

```
if (decoded is! Map<String,dynamic>) {  
  throw const FormatException('Expected a JSON object');  
}
```

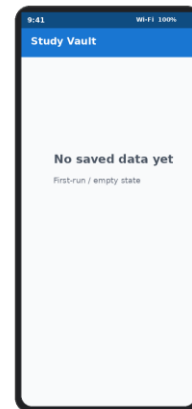
A JSON file fails schema validation. Which labeled screen communicates the error and recovery?



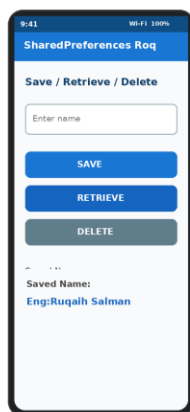
A.



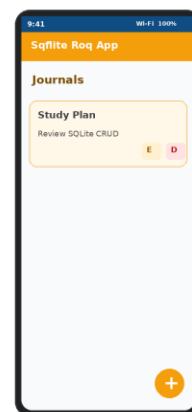
B.



C.



D.

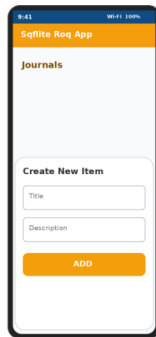
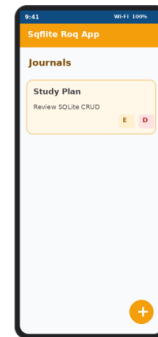
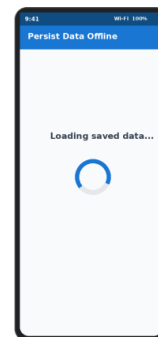


**Q3. Match Code with Screen**

Match every numbered code fragment with the correct labeled screen. Write the matching screen letter beside each code number.

Set 1. SQLite and preference persistence states - match Code 1-4 with Screens A-D.

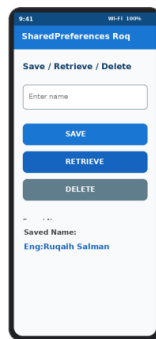
```
Code 1: FloatingActionButton(onPressed: () => _showForm(null), child: const Icon(Icons.add))
Code 2: final data = await SQLHelper.getItems(); setState(() => _journals = data);
Code 3: await SQLHelper.deleteItem(id); await _refreshJournals();
showSnackBar(...);
Code 4: setState(() => _isLoading = true); final data = await
SQLHelper.getItems();
```

**A.****B.****C.****D.**

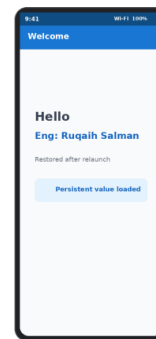
Set 2. File, JSON, restoration, and migration states - match Code 1-4 with Screens A-D.

```
Code 1: final value = await _prefs.getString(_usernameKey) ?? ''; if (mounted)
setState(() => name = value);
Code 2: initState() { super.initState(); _retrieveName(); }
Code 3: final data = await file.exists() ? await file.readAsString() : '';
setState(() => _content = data);
Code 4: if (decoded is! Map<String,dynamic>) throw const FormatException(...);
```

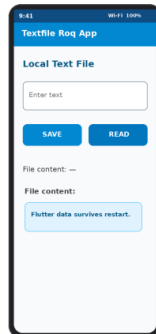
A.



B.



C.



D.

